



Введение в информационные технологии

Лекция №1. Системы контроля версий
и практическая работа с Git

09.03.04 «Программная инженерия»

Потапченко Тимур Дмитриевич · кафедра программной инженерии



Программа курса: первый семестр

Девять тем — от инструментов разработчика до баз данных

1

Системы контроля версий и практическая работа с Git

Как хранить историю проекта и
работать в команде

2

Введение в язык программирования Python

Синтаксис, типы данных, ввод-вывод,
циклы

3

Функции в Python и базовые алгоритмы

Декомпозиция задачи, работа со
списками

4

Организация Python-проекта

Модули, окружения, зависимости,
конфигурация

5

Объектно-ориентированное программирование в Python

Классы, объекты, наследование,
инкапсуляция

Программа курса: первый семестр

Продолжение

6

Разработка GUI-приложений на PyQt

Окна, виджеты, события, компоновка интерфейса

7

Качество и надёжность Python-кода

Стиль, статический анализ, отладка, тестирование

8

Основы операционных систем Linux

Файловая система, командная оболочка, права

9

Реляционные базы данных и СУБД

Модель данных, SQL-запросы, проектирование схемы

Как устроена работа

Лекция — теория и разбор понятий. (Тест в конце каждой лекции)

Лабораторная работа — самостоятельная практика с отчётом и защитой.

Что потребуется от вас

Каждый шаг лабораторной работы подтверждается скриншотом, а понятия и команды нужно уметь объяснить на защите.

Что мы должны понять сегодня

Восемь вопросов, на которые вы сможете ответить к концу лекции

01 Зачем нужен контроль версий

02 Как Git хранит историю проекта

03 Что такое repository, working directory, staging area и commit

04 Как создать историю проекта

05 Как работать с ветками

06 Как посмотреть и отменить изменения

07 Чем Git отличается от GitHub

08 Как Git используется в реальной командной разработке

Знакомая ситуация

Так выглядит проект, у которого нет системы контроля версий

БЕЗ КОНТРОЛЯ ВЕРСИЙ

final.docx

final_2.docx

final_точно.docx

final_точно_последняя.docx

final_точно_последняя(1).docx

Какой файл актуальный? Что изменилось между ними?

С СИСТЕМОЙ GIT

project/

- 83f28a1 Добавлен расчёт среднего значения
- 7bc214f Добавлена обработка ввода
- 54af927 Создана первая версия программы

Один проект. Полная история. Любую версию можно вернуть.

Копии файлов хранят состояния, но не хранят смысл изменений: кто, когда и зачем их сделал.

Система контроля версий

Version Control System, VCS

Система контроля версий — программное обеспечение, которое хранит историю изменений файлов и позволяет управлять разными версиями проекта.

1

Фиксирует состояния проекта

Каждое сохранённое состояние остаётся доступным навсегда

2

Знает, что изменилось

Можно сравнить любые две версии файла построчно

3

Помнит автора и время

Видно, кто и когда внёс конкретное изменение



Git

Распределённая система контроля версий. Полная история проекта хранится локально, на компьютере разработчика.

Основные операции с историей не требуют подключения к серверу.

Основные задачи системы контроля версий

Пять возможностей, ради которых VCS используют в любом проекте

1

История изменений

Все состояния проекта сохранены и доступны в любой момент

2

Сравнение версий

Видно, какие строки добавлены, изменены или удалены

3

Возврат к состоянию

Можно вернуть проект к предыдущей рабочей версии

4

Параллельная разработка

Несколько направлений работы живут одновременно

5

Совместная работа

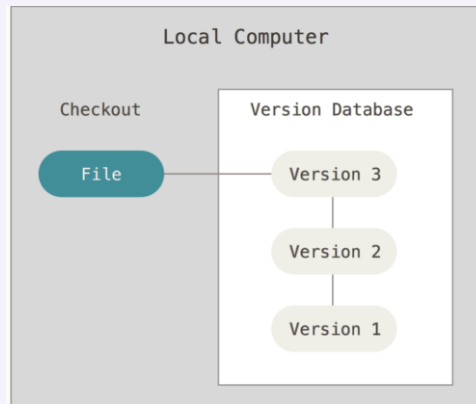
Изменения нескольких человек объединяются без потерь

Все пять задач Git решает локально — даже без доступа в Интернет.

Типы систем контроля версий

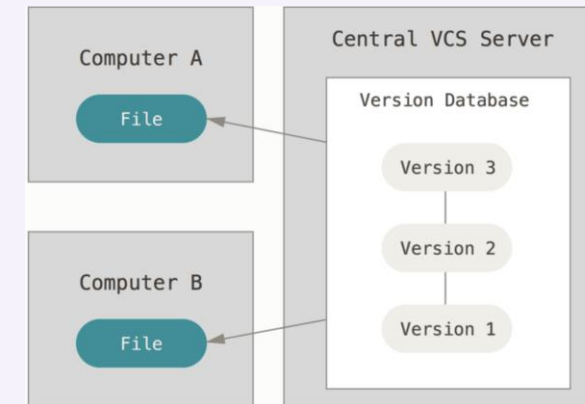
Локальные и централизованные

ЛОКАЛЬНЫЕ



База версий лежит на одном компьютере. Просто, но работать вдвоём невозможно, а потеря диска означает потерю всей истории.

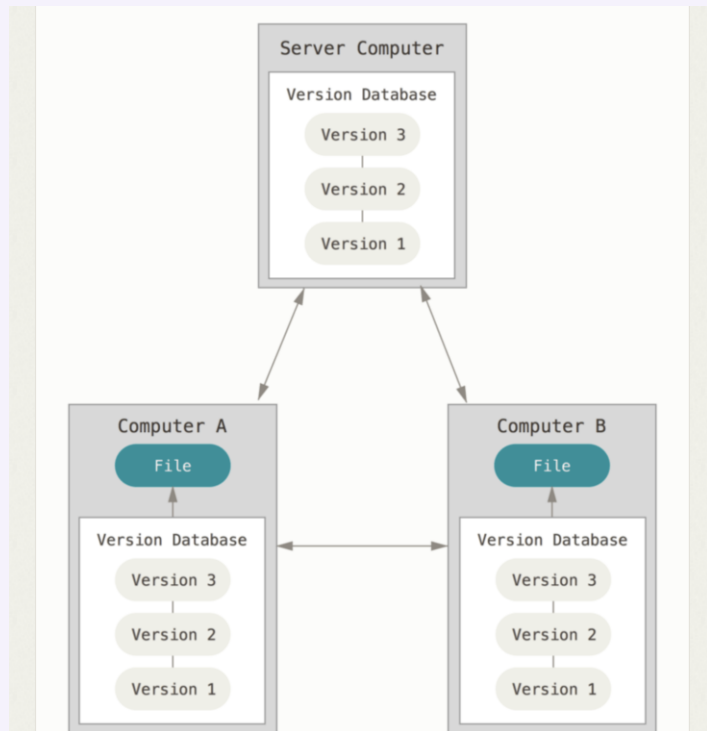
ЦЕНТРАЛИЗОВАННЫЕ



Единый сервер хранит все версии, клиенты забирают нужные файлы. Появилась командная работа, но сервер стал единой точкой отказа.

Распределённые системы контроля версий

Git — распределённая система: у каждого разработчика полная копия репозитория



Полная копия у каждого

Клиент скачивает не снимок файлов, а весь репозиторий вместе с историей

Работа офлайн

Коммиты, ветки, просмотр истории и откат не требуют сети

Устойчивость к сбоям

Любая копия репозитория может восстановить проект целиком

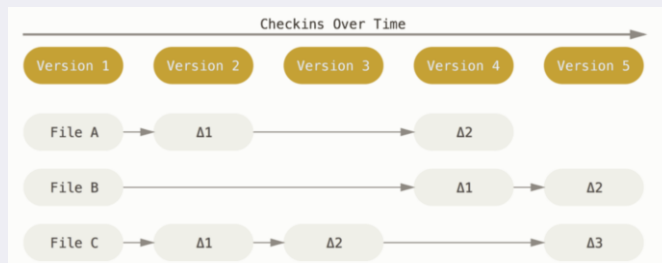
Сервер — точка обмена

Он нужен, чтобы синхронизироваться, а не чтобы работать

Как Git хранит данные

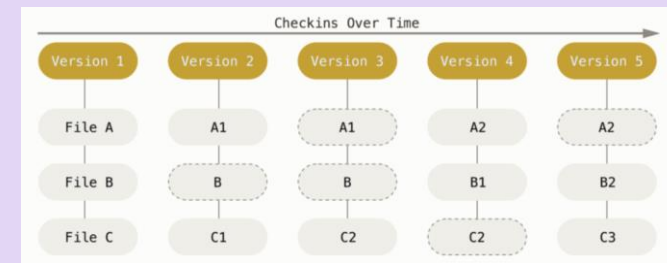
Не набор различий, а последовательность снимков состояния проекта

ДРУГИЕ VCS — НАБОР РАЗЛИЧИЙ



Хранится первая версия файла и цепочка изменений к ней. Чтобы получить версию, различия нужно последовательно применить.

GIT — НАБОР СНИМКОВ



Каждый коммит запоминает, как выглядел проект целиком. Неизменённые файлы не копируются — на них ставится ссылка.

Почему это важно

Git работает не с «историей правок файла», а с состояниями всего проекта. Поэтому переключение веток, сравнение версий и откат выполняются быстро и локально.

Git ≠ GitHub

Это разные вещи: программа на вашем компьютере и сервис в Интернете



Git

система контроля версий

- Устанавливается на компьютер
- Хранит историю в каталоге .git
- Работает полностью локально
- Не требует подключения к Интернету



GitHub

сервис для хранения репозитория

- Работает через Интернет
- Хранит копию репозитория на сервере
- Организует совместную работу команды
- Добавляет задачи, ревью, обсуждения

Git работает независимо от GitHub. GitHub — платформа для размещения Git-репозитория и совместной разработки.

Где размещают Git-репозитории

GitHub — не единственный вариант: протокол и формат репозитория одни и те же



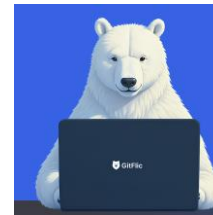
GitHub

Крупнейший сервис,
мировой стандарт
для open source



GitLab

Встроенный CI/CD,
можно развернуть
на своём сервере



GitFlic

Российский сервис
(разработка **МТУСИ**),
используется
в отечественных проектах



Bitbucket

Интеграция
с экосистемой Atlassian
(Jira, Confluence)

Репозиторий, созданный на одном сервисе, без изменений переносится на другой: Git одинаков везде.

Репозиторий

Каталог проекта, историю изменений которого отслеживает Git

Обычная папка становится репозиторием после одной команды. Внутри неё появляется служебный каталог **.git** — именно он и хранит всю историю проекта.

```
git init
```

ЧТО ЛЕЖИТ В .GIT

- **объекты** — все версии файлов и коммитов
- **ссылки** — ветки, теги, указатель HEAD
- **настройки** — конфигурация репозитория

```
git_lab/
├── .git/      ← служебный каталог Git
│   └──      └─ вся история проекта
├── notes.txt
└── main.py
```

Удалить `.git` — значит потерять историю, оставив только файлы.

ЛОКАЛЬНЫЙ

На вашем компьютере.
Полная история проекта.

УДАЛЁННЫЙ

На сервере (GitHub и др.).
Общая точка обмена.

Три области Git

Ключевая схема: как изменение попадает из файла в историю проекта

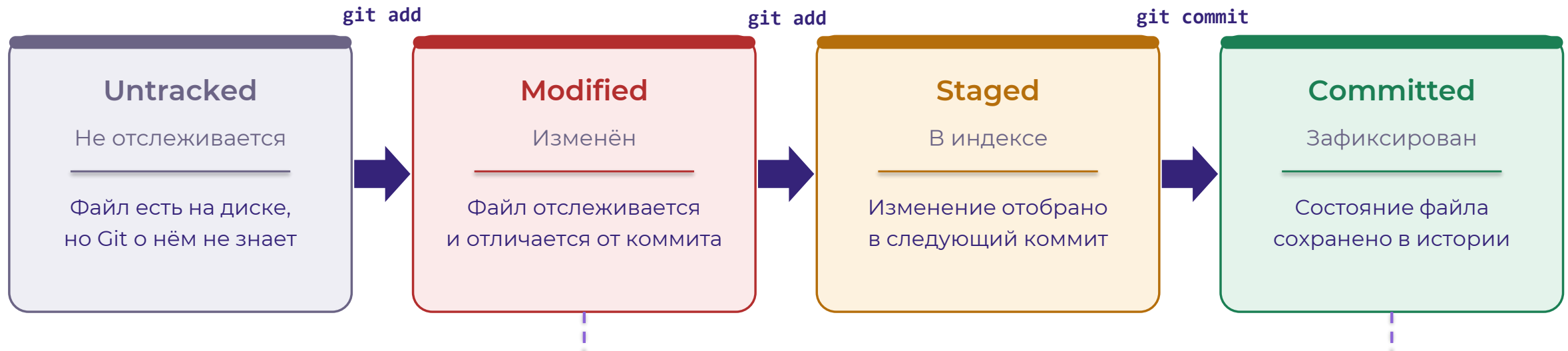


Обратный ход: **git restore** вернёт файл из истории в рабочий каталог, **git restore --staged** уберёт изменение из индекса.

git add отбирает изменения, **git commit** записывает отобранное в историю. Это два разных шага.

Состояния файлов

Файл в репозитории всегда находится в одном из четырёх состояний



После коммита цикл повторяется: файл снова редактируют — он опять становится **Modified**. Из **Staged** изменение можно вернуть обратно командой `git restore --staged`.

Команда `git status` в любой момент показывает, в каком состоянии находится каждый файл.

Коммит — это не Ctrl+S

Коммит фиксирует логически завершённое изменение, а не просто сохраняет файл

Коммит — зафиксированная точка в истории проекта. В него попадает только то, что было отображено в индекс. Каждый коммит знает автора, время, сообщение и предыдущий коммит.

```
git commit -m "Добавлена проверка ввода"
```

Правило

- Один коммит — одна законченная мысль
- Сообщение объясняет, что изменилось и зачем
- Понятно другому человеку и вам через два месяца

ХОРОШО

- ✓ Добавлена проверка пользовательского ввода
- ✓ Исправлена ошибка расчёта среднего значения
- ✓ Обновлено документация по установке

ПЛОХО

- ✗ **изменения** — непонятно, какие именно
- ✗ **123** — не несёт никакого смысла
- ✗ **готово** — что именно готово?

Хеш коммита

Уникальный идентификатор, по которому можно сослаться на любую точку истории

83f28a1d4b9e7c02a5f16be8e3d9c4a17b02e5f9d3c8a41

полный хеш — 40 символов; на практике достаточно первых 7

Вычисляется из содержимого

Хеш зависит от того, что именно сохранено в коммите, и от служебных данных: автора, времени, ссылки на предыдущий коммит.

Изменилось содержимое — изменился хеш

Даже правка одного символа даёт совершенно другой идентификатор. Подменить коммит незаметно не получится.

Служит адресом в истории

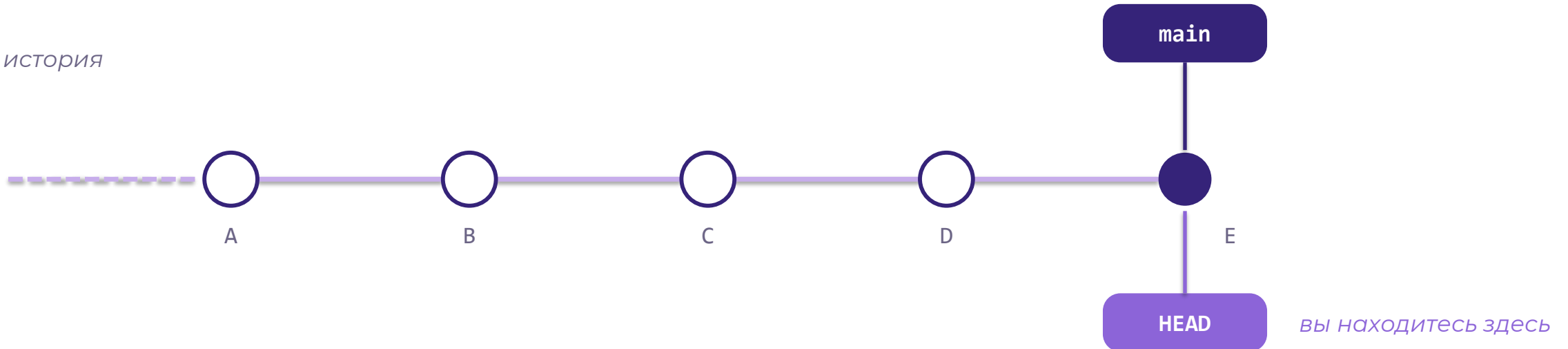
По хешу можно посмотреть коммит, сравнить его с другим или отменить его командой `git revert`.

В командах Git хеш можно указывать сокращённо — достаточно первых 7 символов, если они однозначны.

HEAD

Указатель на то место в истории, где вы находитесь прямо сейчас

история



Что показывает HEAD

Состояние файлов в рабочем каталоге соответствует тому коммиту, на который указывает HEAD.

Куда он перемещается

При переключении ветки или коммита HEAD переходит в новое место — и рабочий каталог меняется вместе с ним.

Как читать HEAD~1

Запись HEAD~1 означает «коммит на один шаг раньше текущего», HEAD~2 — на два шага, и так далее.

Установка и первоначальная настройка

Git должен знать, кто вы: имя и почта попадают в каждый ваш коммит

1

Проверяем, что Git доступен

```
git --version
```

Команда выводит установленную версию. Если версия не выводится — Git не установлен или не добавлен в PATH.

2

Указываем имя и почту

```
git config --global user.name "Иван Иванов"
```

```
git config --global user.email "student@example.com"
```

Флаг --global применяет настройку ко всем репозиториям на этом компьютере.

3

Проверяем настройки

```
git config --list
```

Команда выводит текущую конфигурацию Git, включая user.name и user.email.

Имя и почта — это подпись автора в истории проекта. Указывайте те, по которым вас узнают.

git init — создание репозитория

Превращает обычную папку в Git-репозиторий

```
git init
```

Команда создаёт в текущем каталоге служебную папку **.git**. С этого момента Git начинает следить за проектом: появляется ветка по умолчанию и возможность делать коммиты.

Основная ветка в современных проектах называется **main**. Задать её по умолчанию: `git config --global init.defaultBranch main`

```
$ git init
Initialized empty Git repository in
  /home/student/git_lab/.git/

$ ls -a
.  ..  .git  ← каталог появился
```

ЧТО ПРОИЗОШЛО

- **появился каталог .git** — здесь хранится история
- **текущей веткой по умолчанию становится main**
- **папка стала репозиторием** — можно делать коммиты

git status — что происходит прямо сейчас

Самая часто используемая команда: показывает состояние рабочего каталога и индекса

```
git status
```

```
$ git status
On branch main

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.txt
```

После git add тот же файл выглядит иначе:

```
$ git status
On branch main
Changes to be committed:
  new file:   notes.txt
```

ЧТО ЧИТАТЬ В ВЫВОДЕ

On branch — текущая ветка

Untracked files — Git об этих файлах ещё не знает

Changes not staged — файл изменён, но не в индексе

Changes to be committed — войдёт в следующий коммит

git add — отбираем изменения

Команда не сохраняет проект, а решает, что войдёт в следующий коммит

```
git add notes.txt
```

добавить один файл

```
git add .
```

добавить все изменения
текущего каталога

```
git add src/
```

добавить содержимое папки

Зачем нужен отдельный шаг

Вы правили сразу несколько файлов, но изменения относятся к разным задачам. Индекс позволяет собрать первый коммит только из части правок, а остальные закоммитить отдельно — история останется понятной.

Изменены два файла, в индекс берём только один:

```
$ git add README.md
$ git status
On branch main
Changes to be committed:
  modified:   README.md    ← войдёт в коммит

Changes not staged for commit:
  modified:   app.py       ← останется на потом
```

Один и тот же момент времени

README.md уже отобран и попадёт в коммит, app.py остаётся изменённым в рабочем каталоге. Git видит оба файла, но в историю сейчас уйдёт только первый.

git commit — фиксируем изменения

Записывает содержимое индекса в историю проекта как новую точку

```
git commit -m "Добавлен файл notes.txt"
```

ЧТО ХРАНИТ КОММИТ

- **снимок проекта** — состояние всех файлов на этот момент
- **автор и время** — кто и когда зафиксировал изменение
- **сообщение** — что было изменено и зачем
- **ссылку на предыдущий** — связывает историю в цепочку

В коммит попадает только то, что было отобрано командой **git add**. Правки, оставшиеся в рабочем каталоге, в историю не уходят.

```
$ git commit -m "Добавлен файл notes.txt"
[main (root-commit) d378dda] Добавлен файл notes.txt
1 file changed, 1 insertion(+)
create mode 100644 notes.txt
```

```
$ git log --oneline
d378dda Добавлен файл notes.txt
```

КАК ЧИТАТЬ ВЫВОД

main — ветка, в которой создан коммит

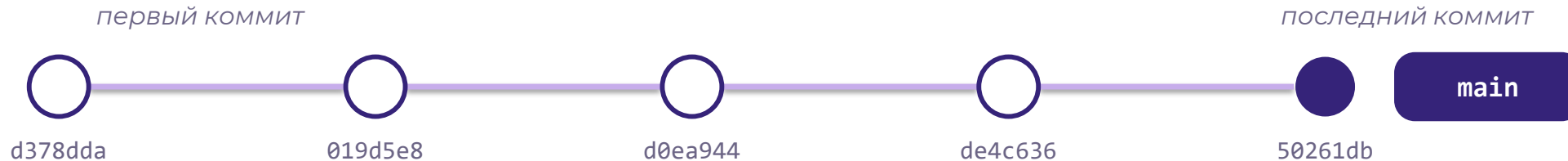
root-commit — это самый первый коммит в истории

d378dda — начало хеша — идентификатор коммита

1 file changed — сколько файлов затронуто

git log — история проекта

Каждый коммит ссылается на предыдущий, образуя цепочку



`git log`

полная запись каждого коммита

```
$ git log
```

```
commit 50261db8f3a1c47e9d2b6f08a3517cc4de91b7a2
```

```
Author: Иван Иванов <student@example.com>
```

```
Date: Mon Sep 1 10:31:14 2025 +0300
```

```
Добавлен файл experiment.txt
```

```
commit de4c6360b17f5a92e4c8d310fb7629ae5c04d8f1
```

```
Author: Иван Иванов <student@example.com>
```

```
Date: Mon Sep 1 10:28:02 2025 +0300
```

```
Добавлена пятая строка
```

`git log --oneline`

по одной строке на коммит

```
$ git log --oneline
```

```
50261db Добавлен файл experiment.txt
```

```
de4c636 Добавлена пятая строка
```

```
d0ea944 Добавлена четвертая строка
```

```
019d5e8 Добавлены вторая и третья строки
```

```
d378dda Добавлен файл notes.txt
```

```
# слева — короткий хеш, справа — сообщение
```

```
# порядок: от самого свежего к самому первому
```

Просмотр изменений: git diff

Две команды отвечают на два разных вопроса

`git diff`

Что я изменил, но ещё **не добавил** в индекс.

Сравнивает рабочий каталог с индексом.

```
--- a/notes.txt
+++ b/notes.txt
@@ -1,3 @@
Первая версия файла
+Вторая строка
+Третья строка
```

`git diff --staged`

Что **уже отобрано** и войдёт в коммит.

Сравнивает индекс с последним коммитом.

```
--- a/notes.txt
+++ b/notes.txt
@@ -1,3 @@
Первая версия файла
+Вторая строка
+Третья строка
```

`git diff`

`git diff --staged`

Working Directory

Staging Area

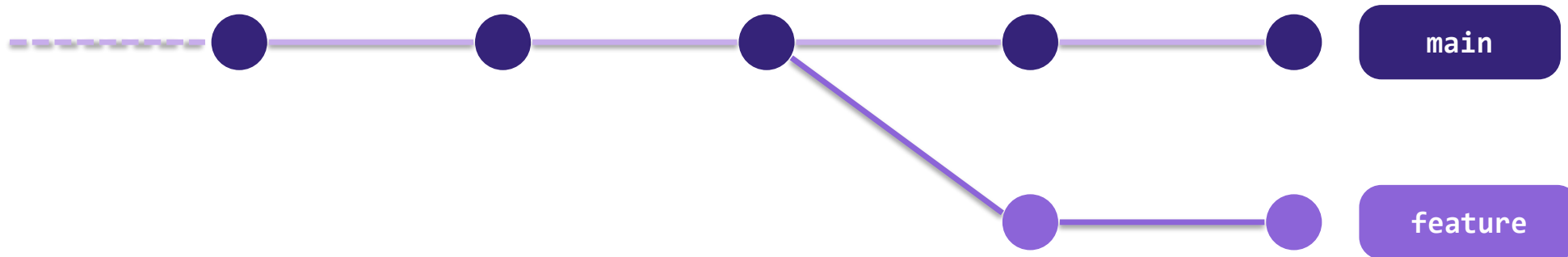
Repository

Ветка — это указатель, а не копия папки

Создание ветки не копирует файлы проекта: Git лишь ставит новую метку на коммит

общая история

указатель на коммит



Ветка — это ссылка

Она хранит всего один хеш — идентификатор коммита, на который указывает. Поэтому создаётся мгновенно.

Переключение меняет файлы

При переходе в другую ветку Git приводит рабочий каталог к состоянию того коммита, на который она указывает.

Основная ветка — main

Она содержит стабильную версию проекта. Эксперименты и новые задачи ведут в отдельных ветках.

Работа с ветками

Четыре команды, которых достаточно для повседневной работы

```
git branch
```

показать список веток;
текущая отмечена звёздочкой

```
git branch feature
```

создать ветку feature,
оставшись в текущей

```
git switch feature
```

перейти в ветку feature

```
git switch -c feature
```

создать ветку feature и сразу
перейти в неё

В старых руководствах встречается **git checkout**.
Начиная с Git 2.23 для переключения веток
используется отдельная команда **git switch** — она
понятнее и делает ровно одно действие.

```
$ git switch -c experiment
```

```
Switched to a new branch 'experiment'
```

```
$ git branch
```

```
* experiment      ← текущая ветка
main
```

```
$ git switch main
```

```
Switched to branch 'main'
```

```
$ ls
```

```
notes.txt        ← experiment.txt здесь нет
```

```
$ git branch
```

```
experiment
```

```
* main
```

Объединение веток: git merge

Работа из отдельной ветки возвращается в основную линию проекта



```
git switch main
git merge experiment
```

Порядок действий

- Переходим в ветку, куда вливаем изменения
- Выполняем merge с именем ветки-источника

Если обе ветки развивались независимо, Git создаёт отдельный merge commit — его рассмотрим как другой сценарий.

```
$ git merge experiment
Updating de4c636..50261db
Fast-forward ← новый коммит не создан
 experiment.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 experiment.txt

$ git log --oneline
50261db Добавлен файл experiment.txt
de4c636 Добавлена пятая строка
```

Конфликт слияния

Git не может решить за человека, какой из двух вариантов правильный

Конфликт — не поломка Git. Он возникает, когда одни и те же строки файла изменены по-разному в двух ветках. Git сохраняет оба варианта и просит вас выбрать.

Так конфликт выглядит в файле:

```
$ git merge feature
CONFLICT (content): Merge conflict in app.py

<<<<<< HEAD      ← ваша версия (main)
print("Привет")
=====
print("Hello, world!")
>>>>>> feature   ← версия из ветки
```

Всё между <<<<<< и ===== — текущая ветка; между ===== и >>>>>> — вливаемая.

КАК РАЗРЕШИТЬ

- 1 Открыть файл, отмеченный как конфликтный
- 2 Выбрать правильный вариант или объединить оба
- 3 Удалить строки-маркеры <<<<<<, =====, >>>>>>
- 4 Сохранить файл и добавить его в индекс
- 5 Завершить слияние коммитом

```
git add app.py
```

```
git commit
```

Отмена изменений: три разных ситуации

Прежде чем отменять, определите, на какой стадии находится изменение

1

**Изменение только
в рабочем каталоге**

```
git restore filename
```

Файл вернётся к состоянию из последнего коммита.
Несохранённые правки будут потеряны.

Изменение исчезает

2

**Изменение уже
в индексе**

```
git restore --staged filename
```

Изменение уйдёт из индекса, но останется
в рабочем файле — оно не пропадает.

Изменение остаётся в файле

3

**Изменение уже
в коммите**

```
git revert <commit>
```

Git создаст новый коммит,
отменяющий указанный.
История при этом не переписывается.

Старый коммит остаётся в истории

«Отменить изменение» в Git означает разные действия — результат зависит от того, где это изменение уже успело оказаться.

git restore: вернуть файл и убрать из индекса

Одна команда, два режима — важно понимать разницу

```
git restore notes.txt
```

Отменяет правки в рабочем файле

Файл возвращается к состоянию из последнего коммита. Несохраниённые изменения при этом теряются безвозвратно — Git их нигде не запоминал.

```
git restore --staged notes.txt
```

Убирает изменение из индекса

Само изменение остаётся в рабочем файле — оно просто не войдёт в следующий коммит. Файл снова становится Modified.

```
$ git status --short
M notes.txt      ← файл изменён
```

```
$ git restore notes.txt
$ git status --short
                ← пусто: правка отменена
```

```
$ git add notes.txt
$ git status
Changes to be committed:
    modified:   notes.txt
```

```
$ git restore --staged notes.txt
$ git status
Changes not staged for commit:
    modified:   notes.txt ← правка на месте
```

git revert и git reset

Обе команды «отменяют», но по-разному обращаются с историей проекта

```
git revert <commit>
```

БЕЗОПАСНО

Создаёт новый коммит, который отменяет изменения выбранного. Исходный коммит остаётся в истории — видно и что сделали, и что отменили.



История только растёт. Коллеги увидят обе записи и поймут, что произошло.

```
git reset --hard HEAD~1
```

ОПАСНО

Перемещает указатель ветки на другой коммит и способна изменить уже существующую историю. Незакоммиченные изменения при --hard теряются.



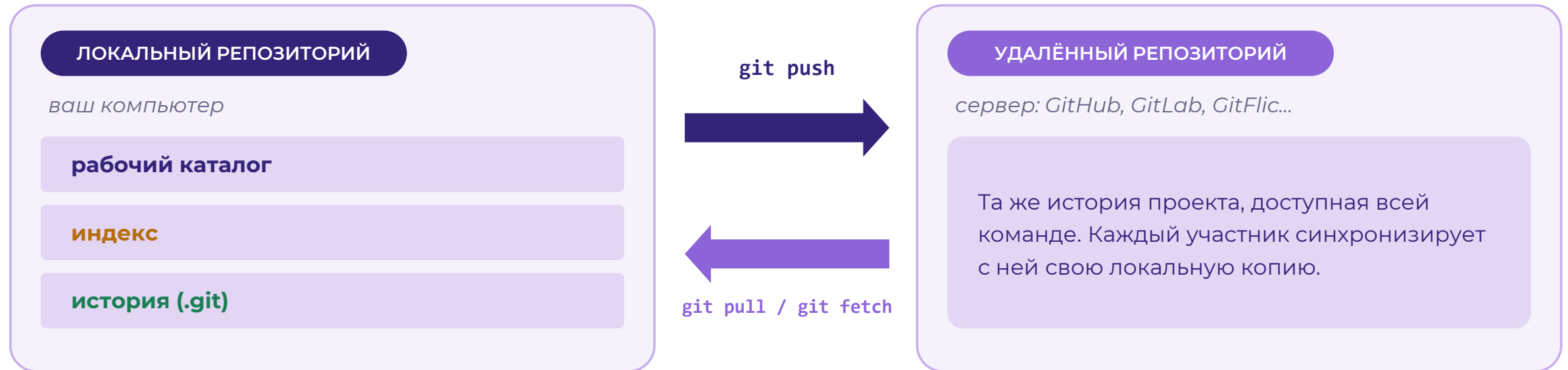
Показываем только как пример опасного действия — в учебной работе не используем.

Почему переписывать опубликованную историю опасно

Коллеги уже скачали старые коммиты. Если история изменилась, их локальные копии расходятся с сервером — и восстанавливать работу приходится вручную.

Локальный и удалённый репозиторий

До сих пор Git работал только на вашем компьютере. Теперь подключим сервер



origin

Стандартное имя основного удалённого репозитория. Это просто короткий псевдоним для его адреса.

Локальный репозиторий полноценен сам по себе

Коммиты, ветки и откаты работают без сервера. Удалённый репозиторий нужен только для обмена.

git clone — получить проект целиком

Скачиваются не только текущие файлы, но и вся история проекта

```
git clone <URL>
```

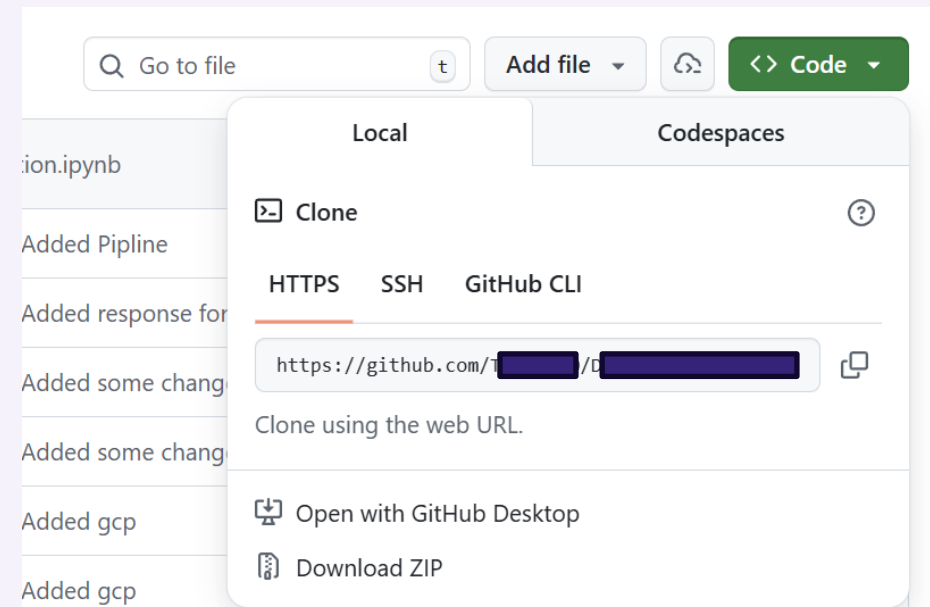
```
$ git clone https://github.com/user/project.git
```

ЧТО ПРОИСХОДИТ ПРИ КЛОНИРОВАНИИ

- скачиваются все файлы проекта
- скачивается вся история коммитов и все ветки
- создаётся каталог .git — репозиторий становится полным
- адрес источника сохраняется под именем origin

После клонирования проект полностью работоспособен офлайн.

Адрес репозитория берут на странице проекта:



git remote -v — проверка связи

Показывает, с какими удалёнными репозиториями связан проект

```
git remote -v
```

```
$ git remote -v
origin https://github.com/user/project.git (fetch)
origin https://github.com/user/project.git (push)
```

ЧТО ВИДНО В ВЫВОДЕ

origin — имя удалённого репозитория

URL — адрес, по которому Git с ним связывается

(fetch) — откуда забираются изменения

(push) — куда отправляются ваши коммиты

Если репозиторий создан локально

После git init удалённого репозитория ещё нет — команда ничего не выведет. Связь добавляют вручную.

```
$ git remote -v
← пусто: связи ещё нет

$ git remote add origin \
  https://github.com/student/git_lab.git

$ git remote -v
origin https://github.com/student/git_lab.git (fetch)
origin https://github.com/student/git_lab.git (push)
```

Одна и та же ссылка используется и для загрузки, и для отправки.

fetch, pull и push

Три команды обмена с удалённым репозиторием

`git fetch`

Забрать сведения об изменениях

Скачивает новые коммиты с сервера и обновляет ссылки вида origin/main. Ваша рабочая ветка при этом не меняется — можно спокойно посмотреть, что нового.

`git pull`

Забрать и внедрить в свою ветку

Скачивает изменения и сразу интегрирует их в текущую ветку. По умолчанию это делается слиянием, но поведение настраивается параметром pull.rebase.

`git push`

Отправить свои коммиты

Передаёт локальные коммиты в удалённый репозиторий. Если на сервере есть коммиты, которых нет у вас, Git отклонит отправку — сначала нужно забрать изменения.

локальный

push →

origin

← fetch / pull

Аутентификация на GitHub

Обычный пароль от аккаунта для операций с репозиторием больше не используется

HTTPS

Токен вместо пароля

При первой отправке Git запросит учётные данные.
Вместо пароля указывается персональный токен доступа (personal access token).

Менеджер учётных данных

Git Credential Manager запоминает токен, поэтому вводить его при каждой операции не требуется.

Проще начать, ничего не нужно настраивать заранее.

SSH

Пара ключей

На компьютере создаётся пара ключей: закрытый остаётся у вас, открытый добавляется в настройки аккаунта.

Без ввода данных

После настройки push и pull выполняются без запроса логина и токена.

Удобнее при постоянной работе с репозиториями.

Что изменилось

Раньше по HTTPS Git спрашивал логин и пароль от аккаунта. Сейчас пароль для этих операций не принимается: нужен токен доступа либо SSH-ключ.

.gitignore — что не должно попасть в репозиторий

Файл со списком того, что Git не должен отслеживать

.gitignore — обычный текстовый файл в корне проекта. Каждая строка описывает, какие файлы и папки Git должен игнорировать.

Типичный .gitignore для Python-проекта:

<code>__pycache__/</code>	кэш интерпретатора
<code>*.pyc</code>	скомпилированные модули
<code>.venv/</code>	виртуальное окружение
<code>.env</code>	секреты: токены, пароли, ключи
<code>*.log</code>	журналы работы программы

.env — потенциально секретные данные. В репозиторий попадать не должен.

ЗАЧЕМ ЭТО НУЖНО

- репозиторий не засоряется временными файлами
- секреты не попадают в общую историю проекта
- чужие окружения и кэш не конфликтуют между собой
- в изменениях видно только то, что писал человек

А как быть с .vscode/ ?

Однозначного ответа нет. Личные настройки редактора обычно игнорируют. Но часть файлов — например, общие настройки форматирования или конфигурацию запуска и отладки — команды сознательно хранят в Git, чтобы проект одинаково открывался у всех.

.gitignore не действует задним числом

Файл, который уже попал в репозиторий, продолжит отслеживаться

ПРОБЛЕМА

Файл **.env** был добавлен командой `git add` ещё до того, как появился `.gitignore`. Git уже отслеживает его, и правило игнорирования на такой файл не распространяется — изменения продолжают попадать в коммиты.

РЕШЕНИЕ

```
git rm --cached .env
```

Команда убирает файл из отслеживания, но оставляет его на диске. После этого нужно сделать коммит.

```
$ git rm --cached .env
rm '.env'
```

```
$ git status --short
```

```
D .env          убран из отслеживания
```

```
?? .gitignore   новый файл правил
```

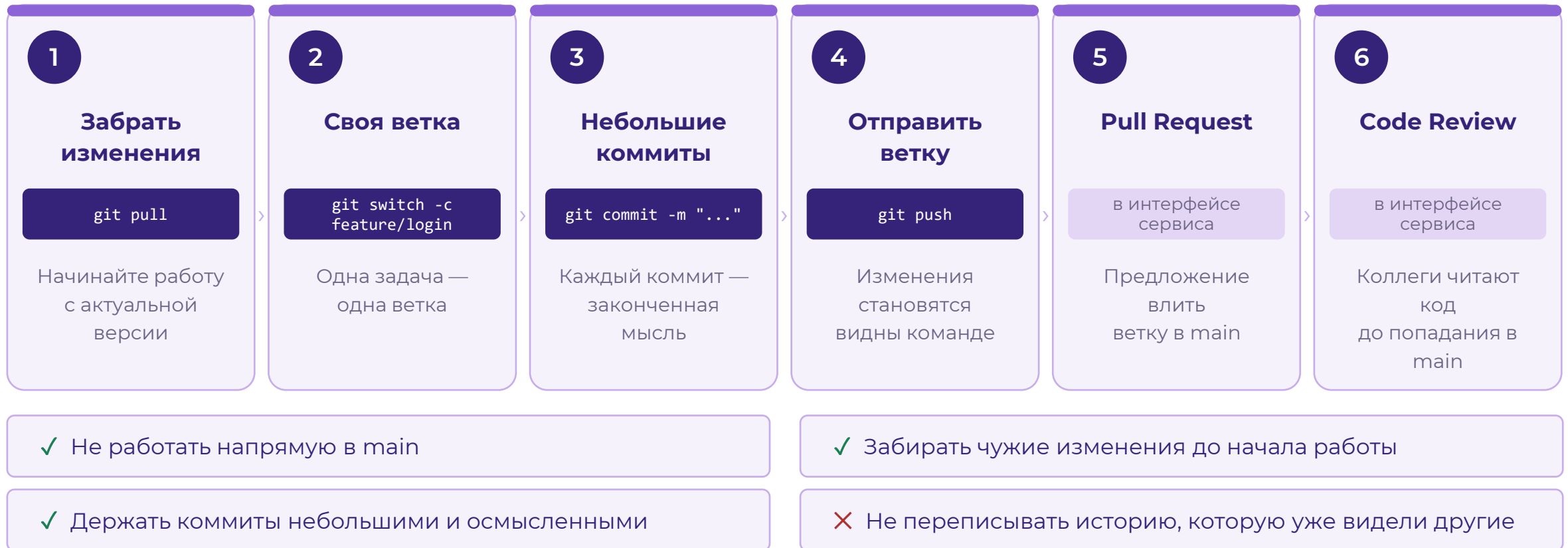
```
$ git commit -m "Убран .env из репозитория"
```

Важно помнить

Секрет, который уже попал в историю, остаётся в старых коммитах. Такой токен или пароль нужно считать скомпрометированным и заменить.

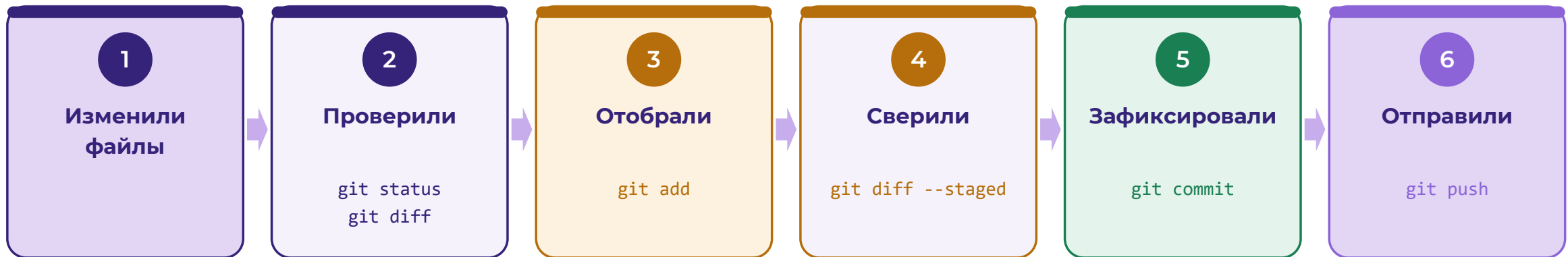
Базовые правила командной разработки

Как выглядит один рабочий цикл в реальном проекте



Рабочий цикл Git

Последовательность, которая повторяется каждый рабочий день



ЕСЛИ ЗАДАЧА ОТДЕЛЬНАЯ

`git switch -c feature``> работа и коммиты``> git switch main``> git merge feature`

Если что-то пошло не так

```
git restore · git restore --staged  
git revert <commit>
```

Привычка выполнять **git status** после каждого шага снимает большую часть вопросов на старте работы с Git.

Git: что необходимо запомнить

Команды, которых достаточно для уверенной повседневной работы

НАЧАЛО РАБОТЫ

git --version

проверить, что Git установлен

git config --global user.name "..."

указать имя автора

git config --global user.email "..."

указать почту автора

git config --list

посмотреть настройки

git init

создать репозиторий

git clone <URL>

клонировать существующий

ЕЖЕДНЕВНЫЙ ЦИКЛ

git status

состояние рабочего каталога

git diff

изменения вне индекса

git diff --staged

изменения в индексе

git add <file> · git add .

отобрать в коммит

git commit -m "..."

зафиксировать изменения

git log --oneline

посмотреть историю

ВЕТКИ И ОТМЕНА

git branch

список веток

git switch <name> · -c

перейти · создать и перейти

git merge <name>

вливать ветку в текущую

git restore <file>

отменить правку в файле

git restore --staged <file>

убрать из индекса

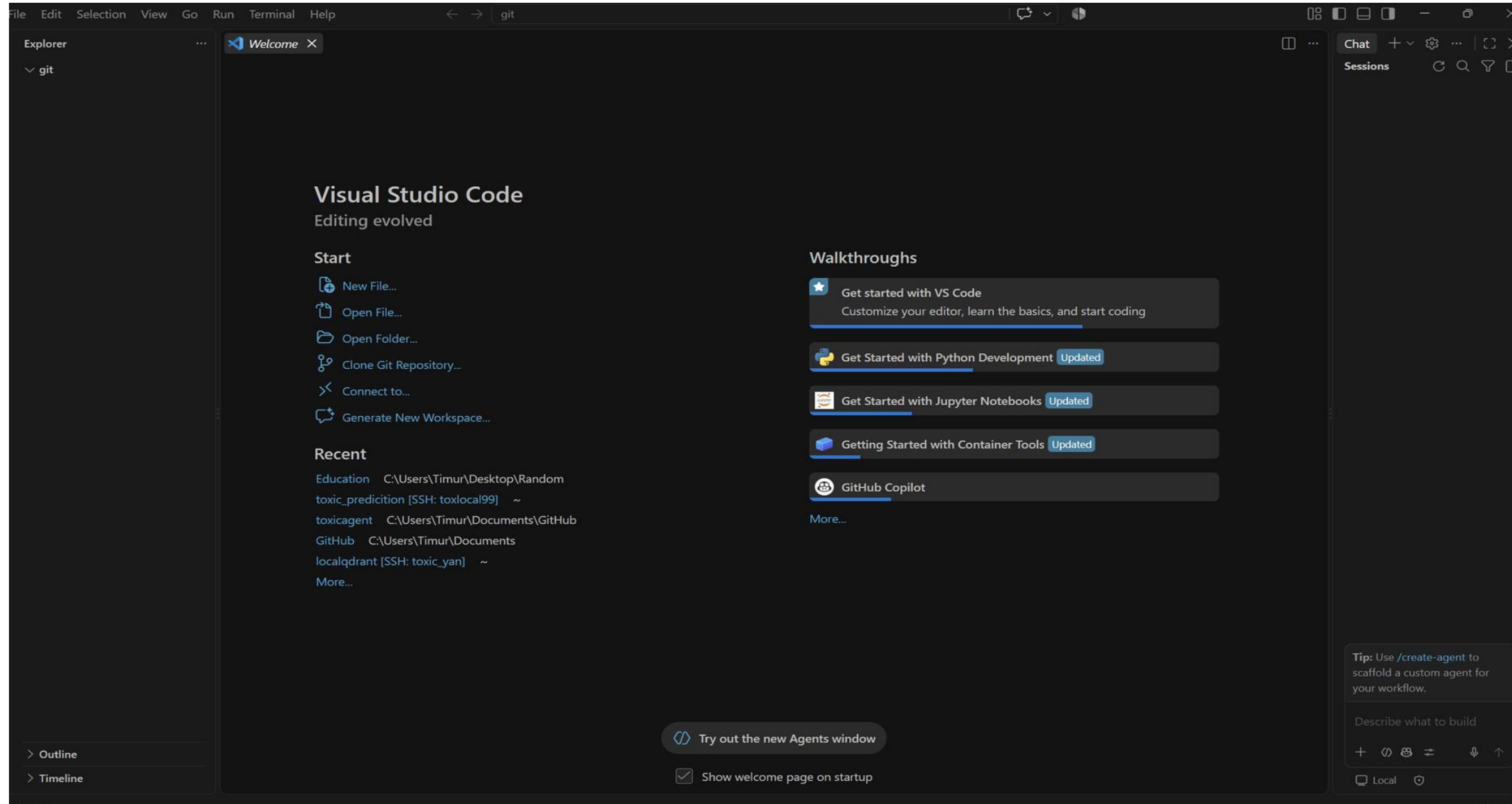
git revert <commit>

безопасно отменить коммит

Справка по любой команде: **git <команда> --help**

Практическая демонстрация

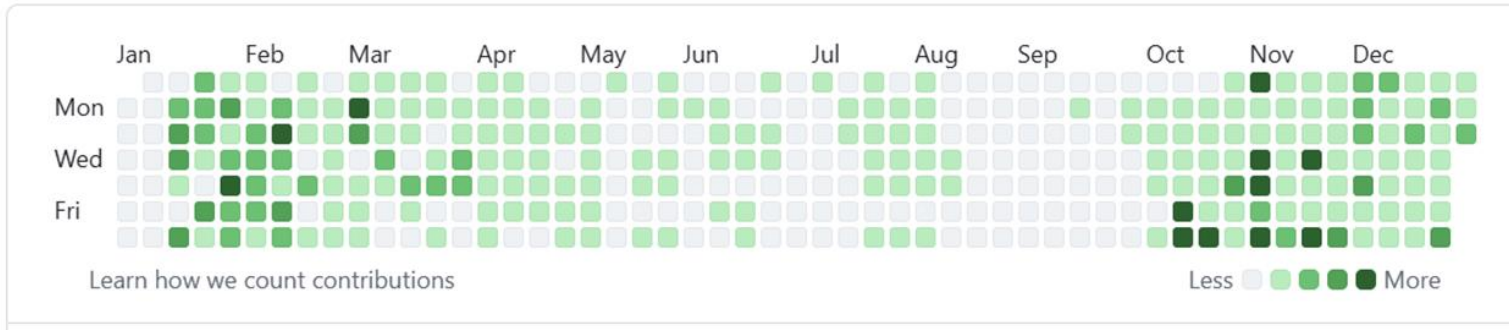
Полный цикл работы с Git в Visual Studio Code





519 contributions in 2024

Contribution settings ▼



СЦЕНАРИЙ ЗАПИСИ

- создание папки проекта
- git init
- создание файла
- git status
- git add
- первый commit
- изменение файла
- git diff
- второй commit
- новая ветка
- изменение в ветке
- git merge
- подключение remote
- git push